

Hands-on Activity 3.1	
Boolean Values, Conditional Execution, Loops, Lists and List Processing, Logical and Bitwise Operations	
Course Code: CPE009A	Program: Computer Engineering
Course Title: Object Oriented Programming 1	Date Performed:1-7-26
Section:CPE12S1	Date Submitted:1-10-26
Name(s):Niel Vincent B. Condino	Instructor: Engr. Jimlord M. Quejado
6. Output	

```
#Leap year or Common Year checker

year = int(input("Enter a year: "))
messages = ["Not within the Gregorian Calendar","Common year","Leap year"]

#if the year number isn't divisible by four, it's a common year;
#otherwise, if the year number isn't divisible by 100, it's a leap year;
#otherwise, if the year number isn't divisible by 400, it's a common year;
#otherwise, it's a leap year.

if year < 1582:
    print(messages[0])
elif (year % 4) > 0: #not divisible by 4
    print(messages[1])
elif (year % 100) > 0:
    print(messages[2])
elif (year % 400) > 0:
    print(messages[1])
else:
    print(messages[2])
```

Figure 1.0: Python Code of “Leap Year checker”

```
Enter a year: 2000
Leap year

Enter a year: 2015
Common year

Enter a year: 1999
Common year

Enter a year: 1996
Leap year

Enter a year: 1580
Not within the Gregorian Calendar
```

Figure 1.1: “Leap Year checker” outputs

Figure 1.0 shows how the leap year checker works. It first takes an input from the user for a year. After that it goes through a series of if checks. If the year is less than 1582, it outputs that it is not within the Gregorian calendar. If not, it checks if it is divisible by four. If it is, then it is a leap year, if not it checks if it is divisible by 400. If it is, then it is a leap year. If the conditions before are not met, it prints a message that says it is a common year. The modulo operator is used to check if the number is divisible because this operation is similar to division but it returns the remainder instead of the quotient. If a number is divisible then there should be no remainder. Storing the messages inside of the list is just for the ease of access so if the programmer chooses to change one message, it should automatically apply on all of the functions in the scope and prevent the work of copy pasting it over and over again. Figure 1.1 shows the different outputs of the code based on the given input

```
import time

for count in range(1,6):#1-6 to repeat 5 times due to the logic of range func
    print(count,"Mississippi")
    time.sleep(1)#"wait" function

print("Ready or not, here I come!")

1 Mississippi
2 Mississippi
3 Mississippi
4 Mississippi
5 Mississippi
Ready or not, here I come!
```

Figure 2.0: "Mississippi loop" Python Code and output

Figure 2.0 shows the code that counts "mississippily". It first imports the time library for the "wait" function later. It then enters a "for" loop that loops 5 times. The lower range given is 1 so it starts with 1 and then the upper range given is 6 so that it loops 5 times due to the behaviour of the range function, it still thinks it starts from 0. It then prints the number of iterations with the word "Mississippi". The code is then yielded using a kind of "wait" function, this being `time.sleep()`. After the loop is finished, it prints a farewell message. This figure also presents the code's output when executed.

```

#Block pyramid height finder
height = 0
base = 0 #Needed no. of box for the base of the pyramid
blocks = int(input("Input number of blocks: "))

while blocks > 0:
    base += 1
    for block in range(1,base + 1): #base +1 for expected behaviour e.g if base = 6, loop 6 times not 5
        blocks -= 1
        if blocks >= 0 and block == base: #check if completed the base
            height += 1
        elif blocks < 0:#not counted if not completed the base
            break

print("The height of the pyramid:",height)

```

Figure 3.0:"Box Pyramid Height checker" Code

```

Input number of blocks: 6
The height of the pyramid: 3

```

```

Input number of blocks: 20
The height of the pyramid: 5

```

```

Input number of blocks: 1000
The height of the pyramid: 44

```

```

Input number of blocks: 2
The height of the pyramid: 1

```

Figure 3.1 "Box Pyramid Height check" Code's Output

Figure 3.0 presents the code that calculates the ideal height of a block pyramid given the number of blocks. After initializing the base and height as 0, it takes an input of the number of blocks. It enters a "while" loop that runs until there are no blocks left. It increments the base by 1 then enters a "for" loop that repeatedly decrements a block for each of it laid in the base. If there are blocks left and the base is complete then it increments the height by one then repeats the logic of the "while" loop. If there are no blocks left and the base is not completed, it is counted to the total height of the pyramid and the loop is broken. After the "while" loop is finished, it prints the height of the pyramid of blocks. Figure 3.1 presents the results of the code based on the different inputs. If the number of blocks can create a pyramid that has its base one block greater than the layer above (e.g. 6(1+2+3)) then it prints its completed height. If the number did meet this condition (e.g. 2 (1+1)) then it just prints the height of the completed base which is one block greater than the layer above.

```
#Remove duplicate numbers (loop approach)
my_list = [1, 2, 4, 4, 1, 4, 2, 6, 2, 9]
seen_elem = []
dup_index = []

for i,elem in enumerate(my_list):
    if elem in seen_elem:
        dup_index.append(i)
    else:
        seen_elem.append(elem)

for i in reversed(dup_index):#pop from the highest index in order to avoid index shifting
    my_list.pop(i)

print("The list with unique elements only:")
print(my_list)

The list with unique elements only:
[1, 2, 4, 6, 9]
```

Figure 4.0: “list duplicate checker” code and output using the loop approach

```
#Remove duplicate numbers (set approach)
my_list = [1, 2, 4, 4, 1, 4, 2, 6, 2, 9]

my_list = list(set(my_list))#Turn to set to remove duplicates

print("The list with unique elements only:")
print(my_list)

The list with unique elements only:
[1, 2, 4, 6, 9]
```

Figure 4.1: “list duplicate checker” code and output using the set approach

Figure 4.0 and 4.1 presents 2 code that has similar output while using 2 different approaches. Figure 4.0 loops through the first list. enumerate function is used to get the index and the value of each element in the list. If it has not seen the element then it appends it to the seen_elem list. If it has seen it, then it appends its index to the dup_index list. After this loop, it loops through the dup_index list in the reverse order. This is done in order to avoid the switching of index mid-popping (e.g. popping index 1 causes the indexes next to it to shift left which will cause unexpected behaviour). After this, it prints my_list with unique elements only. Figure 4.1 is a much simpler approach, it converts it to a set then back to a list. To put it simply, a set only stores unique elements in an unordered way which is useful when dealing with duplicate numbers. Similar to 4.1, at the end it prints the list with just unique elements.

7. Supplementary Activity

```
#Student Score Evaluator

#inputted scores
scores = [85, 72, 90, 60, 78, 55, 88]
bonus_flags = [1, 0, 1, 0, 1, 0, 1]

#in code lists
passed_num = 0
failed_num = 0
certificate_num = 0
results = []

for student_num,score in enumerate(scores):#enumerate() to also get the index
    passed = bool(score >= 75)
    remark = ""
    has_bonus = bool(bonus_flags[student_num] & 1)

    if passed:
        remark = "PASSED"
        passed_num += 1
        if has_bonus:
            certificate_num += 1
    else:
        remark = "FAILED"
        failed_num += 1

    results.append("STUDENT " + str(student_num+1) + ": " + remark)

print("Total Student: ", len(scores))
print("Passed: ",passed_num)
print("Failed: ",failed_num)
print("Certificates Earned:",certificate_num)

Total Student:  7
Passed:  4
Failed:  3
Certificates Earned: 4
```

Figure 5.0: “Student Score Evaluator” Code and Output

Figure 5.0 shows the code that evaluates the score of a student and its output. Given the scores and bonus flags of the student, The counter for each student that passed, failed, has certificates. It then loops through the scores list. As used before, `enumerate()` is used to simultaneously get the index and the value of each element in the list. It checks if the student passed by evaluating if its score is greater or equal than 75, it then stores it inside the `passed` variable. The `has_bonus` checks bitwise if the bonus flag is 1 which means the student has a bonus, qualifying them to get a certificate if they have a passing score. If the student has passed then it sets the `remark` value as “PASSED”, increments the counter of those who passed. It then checks if the student has a bonus, if he has, the number of earners of certificate is incremented. If the student did not pass, the `remark` is set as “FAILED” and increments the number of failed students by one. It then appends the result in the result list in the format of “STUDENT (n): ‘remark’”. After this loop is finished, The total of students, the number of passed, failed, and certificates earned is printed.

```
#Bubble Sort

score_list = [22,24,27,39,55,53,60,41,33]
swapped = True #To Enter the bubble sort loop

while swapped:
    swapped = False
    for i in range(len(score_list) - 1):
        if score_list[i] > score_list[i + 1]:
            swapped = True
            score_list[i], score_list[i + 1] = score_list[i + 1], score_list[i]

print(score_list)

[22, 24, 27, 33, 39, 41, 53, 55, 60]
```

Figure 6.0 “Bubble Sort” Code and Output

Bubble Sort is used to sort the elements of an array in ascending or descending order. The code in Figure 6.0 uses a swapped variable that is set to true at first to enter the bubble sort loop. This loop checks if a swap occurred. The algorithm works by comparing the element and the one next to it. If the element is greater than the next one to it, then it swaps the elements' index. This works in multiple passes to ensure that the elements are arranged correctly. If there is no swap that occurred, it stops the loop in order to save memory and execution time.

8. Conclusion

In this activity, I am able to apply my learnings on C++ with the different syntax of python. The same as C++, python has its if, else if, else conditional statements. Python also has lists which behave like an array on C++ without having a fixed size and it accepts different data types in one list. Python also has their for and while loop but it lacks the do while loop of C++. Since these processes have the same core idea as the one in C++, it is easy to learn. By doing these different tasks I was also able to learn different new builtin functions in python. In one of the tasks, I used the reverse() function when popping different elements so that I can avoid the problem of shifted indexes. I also learned of the enumerate function so that I can simultaneously get the index and element in just one “for” loop. I am also able to learn about how sets in python only accept unique elements. In supplementary activities, I find them easy to implement. I learned on one of these how to compare bitwise. I find this kind of comparison pretty interesting as it seems to use 1 and 0. The second supplementary is just bubble sort, as I already know its concept, I just translated my C++ code into python. Overall, I think I did alright in this activity, although there are still areas of improvement. I think I need to improve my debugging and problem solving skills more as it took me like 15 minutes to debug some of my logical errors in code. I think I can improve this by learning more about the programming language of python and becoming more experienced on it.

9. Assessment Rubric